1 Introduction [15 points]

- Group members: Baaqer Farhat and Andrey Korolev
- Kaggle team name: Hood Research Department
- Ranking on the private leaderboard: 18 (top 50%)
- AUC score on the private leaderboard: 0.84087
- Colab link: `TBD`
- Piazza link: `TBD`
- Division of labor: We divided the work equally between both group members. We collaborated on developing and evaluating multiple models, tuning their hyperparameters, and combining the strongest models into a final ensemble that produced our best predictions.

2 Overview [15 points]

Models and techniques attempted

We started with L2-regularized logistic regression as a simple baseline. We then trained CatBoost, LightGBM, and XGBoost models because boosted trees can capture nonlinear relationships between the audio features. We also tested Random Forest, Extra Trees, Histogram Gradient Boosting, and an Explainable Boosting Machine. The weaker models were not included in the final prediction.

We tested probability blending, cross-fitted ensemble weights, repeated fold-and-seed bagging, and rank averaging. We also modified the data by adding release-date features, audio-feature interactions, track frequency (URI count), and album-level features. One less standard feature used the nearest tracks with the same release date in normalized audio-feature space. This was a fold-safe k-nearest-neighbor feature, not a separate KNN classifier; its construction is detailed in Appendix Section A.3. Our final ensemble rank-averaged predictions from selected LightGBM, XGBoost, CatBoost, and Extra Trees configurations trained across multiple grouped folds and random seeds. We also tested release-period specialist models, but only retained changes that improved grouped validation. Appendix Section A.1 explains the role of each retained model.

Work timeline

- **Monday:** Read the project requirements, inspected the data, selected the AUC metric, and planned grouped validation by track URI.
- **Tuesday:** Built the logistic-regression baseline and its preprocessing pipeline.
- **Wednesday:** Trained and tuned CatBoost, then reviewed fold scores and feature importance.
- **Thursday:** Trained and tuned LightGBM and XGBoost and compared their out-of-fold predictions.
- **Friday:** Tested Random Forest, Extra Trees, Histogram Gradient Boosting, EBM, and several weighted ensembles.
- **Saturday:** Added release-date, URI-count, album-level, interaction, and same-date nearest-neighbor features. We repeated grouped cross-validation over several seeds and began testing submissions on Kaggle.
- **Sunday:** Built the final four-model rank-average ensemble, tested release-period specialists, continued testing submissions, selected our final prediction, and produced report figures and result tables.

3 Approach [20 points]

Data exploration, processing and manipulation

We inspected missing values, class balance, repeated URIs, date formats, and each feature's relation to popularity.

Group	Processing and purpose
Cleaning	Mapped High/Low to 1/0; removed missing URIs, links, and the constant type field.
Logistic	Filled missing values with column medians, scaled audio features, and encoded categories.
Tree models	Kept original numeric scales and used native categories when supported.
Date/audio	Added date components, age, log transforms, and interactions to expose timing and nonlinear effects.
Track/album	Added URI count, date-group size, fold-safe date target means, and same-date neighbor scores.

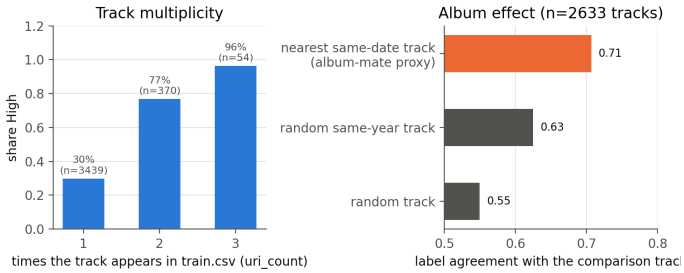


Figure 1: Repeated tracks and same-date neighbors contain strong popularity signals.

Scikit-learn handled missing values, scaling, encoding, and grouped splits. Label-based album features were rebuilt inside every training fold using inner URI-grouped folds to prevent leakage. Near-duplicate indicators were tested but removed because they did not improve validation. Appendix Section A.5 lists the complete feature construction.

Details of models and techniques

Model	Motivation	Limitation or outcome
L2 logistic regression	Fast, interpretable baseline.	Linear boundary missed important interactions.
CatBoost	Native categorical handling and strong nonlinear fits.	Slower to train and sensitive to tuning.
LightGBM/XGBoost	Efficient boosting with strong regularization controls.	Could overfit without careful depth, sampling, and tree-count tuning.
Random Forest/Extra Trees	Bagged trees added prediction diversity.	Random Forest was weaker; Extra Trees was retained in the final blend.
Other alternatives	Tested Histogram Boosting, EBM, SVM, KNN, and spline logistic regression.	Their validation scores were too low to improve the blend.

We measured AUC with stratified cross-validation grouped by URI. We tuned depth, learning rate, tree count, sampling, and regularization through manual searches and Optuna using out-of-fold scores. The final run used ten folds and five seeds. Among probability blends, cross-fitted weights, release-period specialists, and rank averaging, the four-model rank average was most stable. It combined selected LightGBM, XGBoost, CatBoost, and Extra Trees configurations, at the cost of more training and lower interpretability. Exact settings and the full combination procedure are reported in Appendix Sections A.4 and A.6.

4 Model Selection [20 points]

Scoring

Each model predicted the probability that a track belonged to the `High` class. Logistic regression and the boosting models were trained with binary cross-entropy, also called log loss, because it penalizes incorrect and overconfident probabilities. Regularization limited overly complex coefficients or trees.

We selected hyperparameters and ensembles using ROC AUC, the Kaggle evaluation metric. AUC measures how often a randomly selected High track is ranked above a randomly selected Low track. It does not require a classification threshold and is appropriate for our unequal class counts. The logistic baseline scored 0.7216, the first tree models reached about 0.83, and the advanced ensemble reached 0.8355 before the final feature and tuning changes.

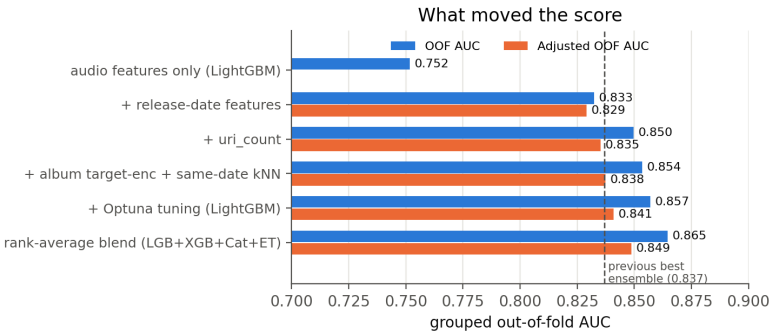


Figure 2: Validation gains from data analysis, feature additions, tuning, and the final ensemble.

Adjusted OOF AUC downweights repeated tracks to match their test-set frequency. LightGBM and XGBoost tied as the strongest individual models at 0.8435 adjusted AUC, followed by CatBoost (0.8413) and Extra Trees (0.8375). Their different errors raised the rank-average ensemble to 0.8487 (Appendix Section A.2).

Validation and test

We first used five-fold stratified cross-validation, then used ten folds and five random seeds for the final comparison. Folds were grouped by URI, so all copies of one track stayed together, while stratification kept similar class ratios across folds. Every row received an out-of-fold prediction. Features based on labels were calculated only from the training portion of each fold. We averaged the fold and seed predictions for the unlabeled competition test set.

Because repeated URIs were more common in training, adjusted OOF gave a more realistic estimate (Appendix Section A.7).

Evaluation	AUC	What it showed
Grouped OOF	0.8646	Best local ranking estimate, but duplicates increased it.
Adjusted OOF	0.8487	More realistic estimate for the competition test set.
Kaggle public leaderboard	0.87013	Strong result on the smaller public split.
Kaggle private leaderboard	0.84087	Final score; rank 18 and top 50%.

5 Conclusion [20 points]

Insights

We ranked features within each final model and averaged those ranks across LightGBM, XGBoost, CatBoost, and Extra Trees. This avoids comparing raw importance values that use different scales. The directions below come from class averages and rank correlations; they describe general trends, not causal effects.

Rank	Feature	Observed influence
1	Same-date neighbor score	Higher scores favored High popularity.
2	Loudness–energy interaction	Important nonlinear interaction; no single direction.
3	Instrumentalness	Higher values favored Low popularity.
4	Release-date position	More recent dates generally favored Low popularity.
5	Log instrumentalness	Higher values favored Low popularity.
6	URI count	Tracks appearing more often favored High popularity.
7	Track age in days	Older tracks favored High popularity.
8	Log track age	Older tracks favored High popularity.
9	Release year	More recent years generally favored Low popularity.
10	Loudness	Louder tracks generally favored High popularity.

Release timing and track repetition were the strongest signals. Album similarity, instrumentalness, and loudness were also important. The largest gains came from understanding these patterns and processing them carefully, rather than only increasing model complexity.

We learned how logistic regression differs from several forms of boosted decision trees and how LightGBM, XGBoost, and CatBoost control tree growth. We also learned to use Optuna to tune depth, learning rate, tree count, sampling, and regularization. We learned that random seeds change fold assignments and model randomness. We repeated ten-fold training with five seeds and averaged their predictions, which reduced variance and made the results more stable. Averaging diverse model families improved ranking stability further; Appendix Section A.6 gives the full procedure.

Challenges

The main challenge was overfitting during both model tuning and leaderboard selection. Our best public score was 0.87013, while the private score was 0.84087. The smaller public split was more variable, and repeatedly reacting to it risked selecting changes that did not generalize. Grouped validation and the adjusted OOF score gave a more realistic estimate.

Repeated URIs were distributed differently between training and test data, which made ordinary validation optimistic. Album features based on labels also created a leakage risk, so they required inner grouped folds. Training many models across folds and seeds was computationally expensive. The leakage controls are summarized in Appendix Section A.7.

With more time, we would reserve an untouched URI-grouped holdout set before tuning, set a fixed submission budget, and use nested cross-validation for the final comparison. We would also identify the duplicate and album structure earlier, since these data issues mattered more than testing many weak models.

6 Extra Credit [5 points]

Why use AUC, and is there a better metric?

AUC measures whether the model ranks High-popularity tracks above Low-popularity tracks across every possible classification threshold. This is useful because the competition asks for probability scores and does not define the cost of a false positive or false negative. It is also less affected by class imbalance than accuracy. Rank averaging worked well partly because it directly preserves the ordering that AUC evaluates.

For this competition, AUC is a reasonable primary metric. In a real application, a different metric could be better depending on the goal. Precision at k would be useful if a service could promote only a fixed number of tracks. Log loss or the Brier score would be better if accurate probabilities, rather than only ranking, were important. F1 score or accuracy would require choosing a threshold and error costs that this project does not provide.

Can our methods or pipelines be parallelized?

Yes. Models from different cross-validation folds, random seeds, and model families are independent and can be trained at the same time. Optuna trials can also run in parallel because each trial evaluates a separate hyperparameter configuration. Within a model, LightGBM, XGBoost, and Extra Trees can use multiple CPU cores, while CatBoost and XGBoost can use a GPU.

Our final run used four model families, ten folds, and five seeds, giving many independent training jobs. Their out-of-fold and test predictions only need to be combined after training. In practice, we would limit concurrent jobs to avoid exhausting memory, oversubscribing CPU cores, or making several jobs compete for one GPU.

What additional data would be useful?

Artist history, follower counts, genre, record label, playlist placements, marketing exposure, and time-stamped stream or chart counts would help explain popularity beyond audio properties. Popularity depends heavily on audience reach and promotion, which cannot be inferred from loudness, tempo, or danceability alone. Time-series data could also show whether a track is steadily growing or experiencing a short-lived spike.

These fields would need a fixed collection date before the prediction target is measured. Otherwise, later stream counts or playlist placements could directly reveal the label and create leakage. We did not use any such external data in this project; these are suggestions for a future dataset.

7 LLM Usage

- Name of LLM(s) Used: ChatGPT
- Components involving the LLM: Colab imports and data loading, Matplotlib figure code, and minor debugging.

Colab imports and data loading

Prompt: *"can u write the colab setup code that installs and imports everything we need, then loads train.csv, test.csv and sample_submission.csv. if the files arent there have it ask me to upload them. keep it simple and dont add a bunch of comments"*

Output integrated into the notebook:

```
%pip install -q lightgbm==4.6.0 xgboost==3.2.0 \
    catboost==1.2.10 scikit-learn==1.7.2

import os
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files
from scipy.stats import rankdata
from sklearn.metrics import roc_auc_score, roc_curve
from sklearn.model_selection import StratifiedGroupKFold

required_files = ['train.csv', 'test.csv',
                  'sample_submission.csv']
missing_files = [name for name in required_files
                 if not os.path.exists(name)]
if missing_files:
    print('Upload train.csv, test.csv, and sample_submission.csv')
    files.upload()

train_data = pd.read_csv('train.csv').dropna(
    subset=['uri']).reset_index(drop=True)
test_data = pd.read_csv('test.csv')
sample_submission = pd.read_csv('sample_submission.csv')
train_labels = (train_data[TARGET] == 'High').astype(int).values
```

How it was used: We retained this setup in the public Colab notebook, fixed the package versions, and used the course-provided CSV files as the only data source.

Figure generation

Prompt: *"can u make the matplotlib code for the figuers in our report. show the high popularity rate by release year with the sample counts and also graph how the OOF and adjusted OOF changed when we added features and the ensemble. use blue and orange and make the labels easy to read"*

Output integrated for the release-era figure:

```
release_years = build_release_date_features(
    train_data)['release_year']
year_bins = [1900, 1980, 1990, 2000, 2010, 2015,
             2018, 2020, 2022, 2023, 2024, 2025]
year_labels = ['<1980', '80s', '90s', '00s', '10-14',
               '15-17', '18-19', '20-21', '2022',
               '2023', '2024']
```

```
year_groups = pd.DataFrame({
    'label': train_labels,
    'era': pd.cut(release_years, year_bins,
                  labels=year_labels, right=False),
}).groupby('era', observed=True)['label'].agg(['mean', 'count'])

figure, axis = plt.subplots(figsize=(9, 4))
axis.bar(year_groups.index.astype(str), year_groups['mean'],
         color=BLUE, width=.62)
for index, (rate, count) in enumerate(
        zip(year_groups['mean'], year_groups['count'])):
    axis.text(index, rate + .02, f'n={count}',
              ha='center', fontsize=8, color=MUTED)
axis.axhline(train_labels.mean(), color=MUTED, ls='--', lw=1)
axis.set(xlabel='Album release year',
         ylabel='Share of High-popularity tracks',
         ylim=(0, 1.08),
         title='Release era strongly separates popularity labels')
axis.grid(axis='x', visible=False)
figure.tight_layout()
```

Output integrated for the validation figure:

```
progression = pd.DataFrame({
    'Experiment': ['Audio only', '+ release date', '+ URI count',
                   '+ album and KNN', '+ Optuna tuning',
                   'Final ensemble'],
    'OOF AUC': [0.7517, 0.8325, 0.8497, 0.8536, 0.8571, 0.8646],
    'Adjusted OOF AUC': [np.nan, 0.8292, 0.8353,
                         0.8375, 0.8410, 0.8487],
})
y_positions = np.arange(len(progression))
axes[0].barh(y_positions + .18, progression['OOF AUC'],
             height=.34, color=BLUE, label='OOF AUC')
axes[0].barh(y_positions - .18, progression['Adjusted OOF AUC'],
             height=.34, color=ORANGE, label='Adjusted OOF AUC')
axes[0].set_yticks(y_positions, progression['Experiment'])
axes[0].set(xlim=(.70, .88), xlabel='ROC AUC',
            title='Validation progression')
axes[0].legend(frameon=False, fontsize=8)
```

How it was used: We checked the plotted values against our saved validation results, adjusted the labels and sizing, and used the resulting figures in both the report and Colab notebook.

Minor debugging

Prompt: *"can u check this grouped cross valdiation loop for leakage? make sure the same URI isnt in train and validation and that the release date target encoding and same-date KNN only use the training fold. also add an assert so we can catch it if it happens"*

Output integrated into the validation loop:

```
folds = list(StratifiedGroupKFold(
    n_splits=N_FOLDS, shuffle=True, random_state=seed
).split(train_features, train_labels, groups=train_data['uri']))

fold_feature_matrices = build_fold_feature_matrices(
    train_features, test_features, train_data, test_data,
    train_labels, folds, seed)

for fold_index, (train_indices, validation_indices) in enumerate(folds):
```

```
    assert not (
        set(train_data['uri'].iloc[train_indices])
        & set(train_data['uri'].iloc[validation_indices])
    )
    fold_train_features, fold_validation_features, \
        fold_test_features = fold_feature_matrices[fold_index]

inner_target_encodings = compute_out_of_fold_target_encodings(
    release_dates[train_indices],
    train_labels[train_indices],
    track_uris[train_indices],
    seed * 10 + fold_index)

validation_target_encodings = compute_smoothed_target_encodings(
    release_dates[train_indices],
    train_labels[train_indices],
    release_dates[validation_indices])

fold_validation_features['same_date_neighbor_label_mean'], \
fold_validation_features['same_date_nearest_neighbor_distance'] = \
    compute_same_date_neighbor_features(
        train_audio_features[train_indices],
        release_dates[train_indices],
        train_labels[train_indices],
        train_audio_features[validation_indices],
        release_dates[validation_indices])
```

How it was used: We incorporated the URI assertion and kept all label-derived feature construction inside the outer fold. We then reran grouped validation to confirm that every row received an out-of-fold prediction.

A Appendix

A.1 Why we chose each model

Model	Reason for testing	Final role
L2 logistic regression	Simple, interpretable baseline for measuring linear signal.	Established a 0.7216 AUC baseline; not ensembled.
LightGBM	Fast gradient boosting with detailed controls for tree growth and regularization.	Strong individual model and final ensemble member.
XGBoost	A second boosted-tree implementation with different split and regularization behavior.	Strong individual model and final ensemble member.
CatBoost	Handles categorical features directly and uses a different boosting procedure.	Added diversity to the final ensemble.
Extra Trees	Randomized, independently fitted trees produce different errors from boosting.	Added bagging diversity to the final ensemble.

Baseline preprocessing

- Numeric audio features: filled missing values with column medians and standardized.
- Mode: kept as a binary feature.
- Key and time signature: one-hot encoded with unknown categories ignored.
- Logistic model: L2 penalty, balanced class weights, and $C = 0.1$.

A.2 Experiment progression

Experiment	Grouped OOF AUC	Adjusted OOF AUC
Audio-only LightGBM	0.7517	—
Add release-date features	0.8325	0.8292
Add URI count	0.8497	0.8353
Add album target encoding and same-date KNN	0.8536	0.8375
Tune LightGBM with Optuna	0.8571	0.8410
Four-model rank-average ensemble	**0.8646**	**0.8487**

Main findings

- Release-date features produced the largest single improvement.
- URI count captured the strong signal from repeated tracks.
- Album and same-date KNN features added smaller but repeatable gains.
- Optuna tuning improved each tree family, while the final ensemble reduced model-specific ranking errors.

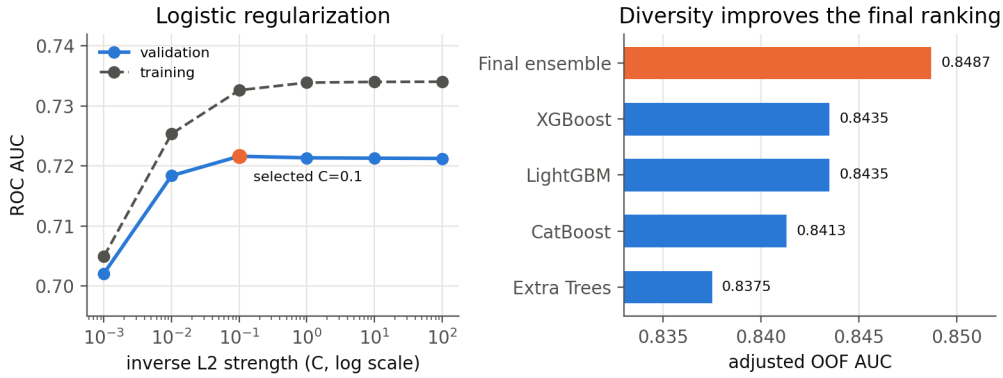


Figure 3: Left: logistic regression was insensitive once C reached 0.1. Right: the rank-average ensemble outperformed every individual model under adjusted OOF AUC.

A.3 Why we used a same-date KNN feature

Motivation

- Exact release dates often group tracks from the same album or release.
- Tracks in one release share artist, promotion, audience, and time period, none of which were directly provided.
- Audio similarity helps distinguish likely album-mates when many tracks share a release date.
- Figure 4 shows why release timing was worth modeling: popularity-label rates changed substantially across release eras.

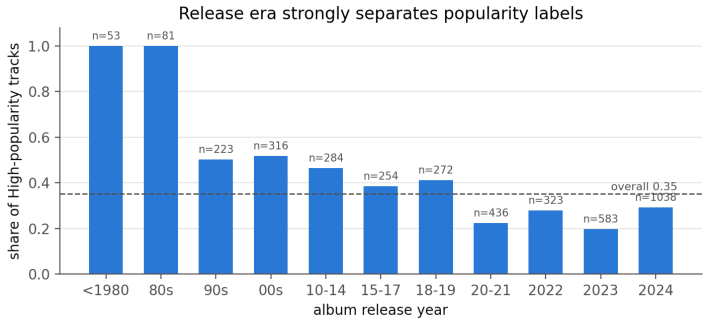


Figure 4: The share of High-popularity labels varied strongly with release era. Counts above each bar show the number of training rows.

Construction

1. Log-transform duration and instrumentalness, then standardize the ten audio features.
2. Restrict candidate neighbors to tracks with the same exact release date.
3. Select the five nearest candidates by Euclidean distance in the normalized audio space.
4. Weight each neighbor by $1/(1+d)$, where d is its distance.
5. Average the neighbor labels with a global-mean prior of weight 2.
6. Supply both the smoothed label mean and nearest-neighbor distance to the tree models.

Why it was a feature, not a KNN classifier

KNN alone relied too heavily on local similarity and ignored broader nonlinear patterns. Supplying the KNN score to the tree models allowed them to combine local album evidence with release age, URI count, audio interactions, and other features.

Leakage prevention

- Validation and test neighbors came only from the outer fold's training rows.
- Training-row scores excluded the row itself and zero-distance copies.
- URI grouping kept all copies of one track in the same outer fold.

Measured contribution

Feature set	Grouped OOF	Adjusted OOF
Through URI count	0.8497	0.8353
Add album target mean and same-date KNN	0.8536	0.8375
Improvement	**+0.0039**	**+0.0022**

A.4 Final model configurations

Tuning process

- Optuna searched tree size, learning rate, row sampling, feature sampling, and regularization.
- Promising settings were rerun under identical grouped folds.
- Selection used adjusted OOF AUC rather than training AUC or a single public leaderboard result.

Model	Final configuration
LightGBM	1,400 trees; learning rate 0.0104; 22 leaves; depth 6; minimum child size 11; row sample 0.555; feature sample 0.572; L1 0.0016; L2 0.0029; extra-trees split mode.
XGBoost	800 trees; learning rate 0.0090; depth 8; minimum child weight 2.46; row sample 0.644; tree feature sample 0.422; L1 0.458; L2 0.271; $\gamma = 0.787$.
CatBoost	800 iterations; learning rate 0.03; depth 6; leaf L2 5; random strength 1; Bayesian bootstrap with temperature 0.5; 128 numeric split borders.
Extra Trees	800 trees; minimum leaf size 3; 50% of features considered per split.

A.5 Additional feature construction

Release-date features

- Year, month, day, and date precision
- Ordinal date, age in days, and log age
- Day of week, day of year, and Friday indicator

Audio transformations

- Log transforms for duration, instrumentalness, speechiness, acousticness, and liveness
- Loudness–energy, danceability–valence, and danceability–energy interactions
- Energy minus acousticness, tempo distance from 120 BPM, and loudness divided by energy

Track and album features

- URI count was calculated separately within train and test.
- Date-group size counted distinct tracks sharing an exact release date.
- Exact-date target means used prior smoothing and inner grouped folds.

A.6 Ensemble construction

Fold and seed bagging

- Ten stratified folds were grouped by URI.
- Seeds 0–4 changed fold assignment and model randomness.
- This produced 50 validation fits per model and reduced dependence on one favorable split.
- Fold and seed test predictions were averaged within each model.

Rank averaging

- Each model's averaged predictions were converted to percentile ranks.
- The four rank vectors were averaged with equal weight.
- Ranking removed differences in probability scale between model families and matched the AUC objective.

A.7 Validation details

Label-derived features

- Date target means and KNN labels were rebuilt inside every outer fold.
- Training-row date means came from an additional five-fold split grouped by URI.
- No validation label was used to create its own features.

Adjusted OOF AUC

- Duplicate URIs represented about 11% of training rows but only 2.5% of test rows.
- Ordinary OOF AUC therefore gave too much influence to repeated tracks.
- Unique rows retained weight 1; duplicate rows were downweighted until their total share matched the test set.
- The adjusted estimate, 0.8487, was closer to the private score of 0.84087 than ordinary OOF or the public score.
- Final changes had to improve grouped and adjusted AUC across multiple seeds; the public score was only supporting evidence.